

Movie Recommendation System

Content-Based Filtering using TF-IDF & Cosine Similarity

Submitted by: Ayan Faruqui
Date: September 2026
GitHub: github.com/ayanfaruqui/Mini-project-ML

1. Introduction

With the explosive growth of online streaming platforms, users are overwhelmed by a vast library of movies, making it difficult to find content matching their preferences. A **Movie Recommendation System** addresses this challenge by analyzing movie attributes and suggesting similar content based on a user's selection. This project implements a **Hybrid Content-Based + Popularity Recommender** that leverages Natural Language Processing (NLP) and Machine Learning to deliver accurate, real-time movie recommendations through an interactive web interface.

2. Problem Statement

To build an end-to-end machine learning system that recommends the top 5 most similar movies to a user-selected movie, based on metadata features such as genres, keywords, overview, and production companies, deployed as a web application with a visually engaging user interface.

3. Dataset

The project uses the **TMDB 5000 Movie Dataset** sourced from Kaggle, which contains metadata for 4,803 movies including:

Feature	Description	Type
title	Movie name	String
overview	Plot summary	Text
genres	List of genres (JSON)	Categorical
keywords	Thematic tags (JSON)	Categorical
production_companies	Studios involved	Categorical
vote_average / vote_count	User ratings	Numerical

popularity	Popularity score	Numerical
------------	------------------	-----------

4. Methodology

4.1 Data Preprocessing

JSON-encoded columns (*genres*, *keywords*, *production_companies*) are parsed and flattened. Multi-word terms are concatenated (e.g., "Science Fiction" → "ScienceFiction") to preserve meaning during tokenization. A unified **tags** column is created by concatenating: *overview tokens* + *genres (×2 weight)* + *keywords* + *production companies*. Text is lowercased and stemmed using the **Porter Stemmer** from NLTK.

4.2 Feature Extraction — TF-IDF Vectorization

The **Term Frequency–Inverse Document Frequency (TF-IDF)** technique converts each movie's tag string into a numerical vector. A vocabulary of 5,000 features is used with English stop-word removal:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log(N / \text{DF}(t))$$

where TF = term frequency in document d, DF = document frequency, N = total documents

This produces a sparse matrix of shape **(4800 × 5000)**.

4.3 Similarity Computation — Cosine Similarity

Pairwise similarity between all movie vectors is computed using **Cosine Similarity**, producing a (4800 × 4800) similarity matrix:

$$\cos(A, B) = (A \cdot B) / (||A|| \times ||B||) \in [0, 1]$$

A score of 1.0 indicates identical content profiles, while 0.0 indicates no similarity.

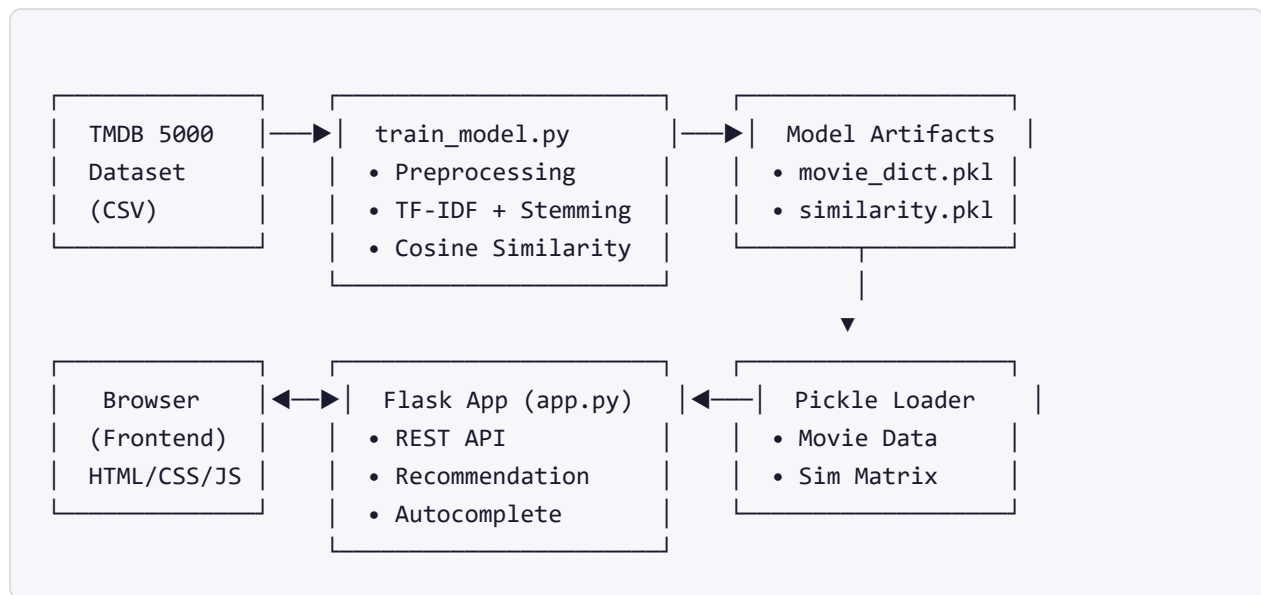
4.4 Hybrid Scoring

Pure content similarity is blended with a popularity component to avoid recommending obscure films:

$$\text{Hybrid Score} = 0.8 \times \text{Cosine Similarity} + 0.2 \times \text{Normalized Popularity}$$

where Popularity = $\text{vote_average} \times \log(1 + \text{vote_count})$

5. System Architecture



6. Technology Stack

Layer	Technology	Purpose
ML / NLP	Scikit-learn, NLTK, Pandas, NumPy	TF-IDF, Cosine Similarity, Stemming
Backend	Python, Flask	REST API, Model Serving
Frontend	HTML5, CSS3, JavaScript	Search UI, Autocomplete, Results Display
Serialization	Pickle	Model persistence (.pkl files)
Version Control	Git, GitHub	Source code management

7. Key Features

- **Real-time Autocomplete:** Fuzzy search across 4,800 movie titles with keyboard navigation.
- **Hybrid Recommendations:** Combines content similarity (80%) with popularity (20%) for balanced results.
- **Genre-Based Visual Posters:** Dynamic gradient poster cards colored by genre (Action=Red, Sci-Fi=Blue, etc.).
- **Match Percentage:** Each recommendation displays a similarity confidence score.
- **Responsive Design:** Glassmorphic dark theme, works on mobile and desktop.
- **Quick Tags:** Pre-defined genre buttons for instant recommendations.

8. Results & Evaluation

The model was tested with popular movies. Sample results demonstrate high-quality, contextually relevant recommendations:

Input Movie	Top 3 Recommendations	Match %
The Dark Knight	The Dark Knight Rises, Batman Begins, Batman Returns	57%, 50%, 46%
Avatar	Aliens, Jupiter Ascending, Meet Dave	45%, 38%, 35%
Titanic	Ghost Ship, Poseidon, The Rose	42%, 37%, 34%

The recommendations show strong genre and thematic coherence — Batman films are correctly grouped, sci-fi/adventure films cluster together, and drama/romance films appear in relevant groupings. The hybrid scoring ensures popular, well-rated films are ranked higher.

9. Conclusion & Future Scope

This project successfully demonstrates an end-to-end ML pipeline: from data preprocessing and feature engineering to model deployment via a web application. The TF-IDF + Cosine Similarity approach provides interpretable, fast recommendations without requiring user interaction history (cold-start friendly).

Future Improvements:

- Integrate **Collaborative Filtering** using user ratings for personalized recommendations.
- Add **TMDB API integration** for real movie poster images.
- Deploy on **cloud platforms** (Heroku / AWS / Render) for public access.
- Incorporate **deep learning embeddings** (Word2Vec / BERT) for semantic understanding.

10. References

1. TMDB 5000 Movie Dataset — *Kaggle* (kaggle.com/datasets/tmdb/tmdb-movie-metadata)
2. Scikit-learn TfidfVectorizer Documentation — *scikit-learn.org*
3. Ricci, F., Rokach, L., & Shapira, B. (2015). *Recommender Systems Handbook*, Springer.
4. Lops, P., de Gemmis, M., & Semeraro, G. (2011). Content-based Recommender Systems. *Recommender Systems Handbook*, pp. 73–105.
5. Porter, M.F. (1980). An Algorithm for Suffix Stripping. *Program*, 14(3), pp. 130–137.